INTECH
PT Industri Teknologi Informasi dan Komunikasi

Pelatihan Software Engineering

# Materi 10
# Merancang Struktur Perangkat Lunak

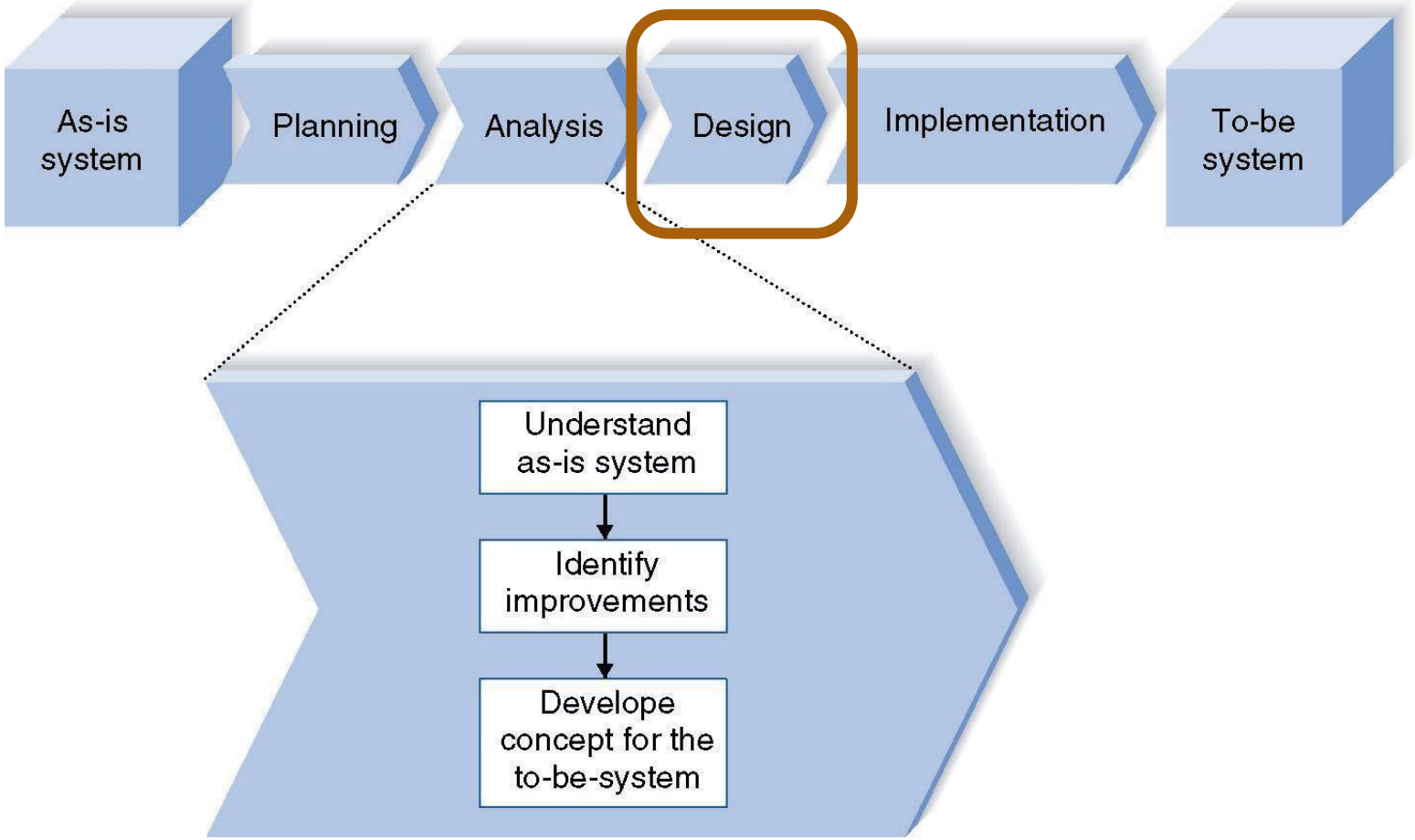


Immah Inayati, S.Kom., M.Kom., MBA

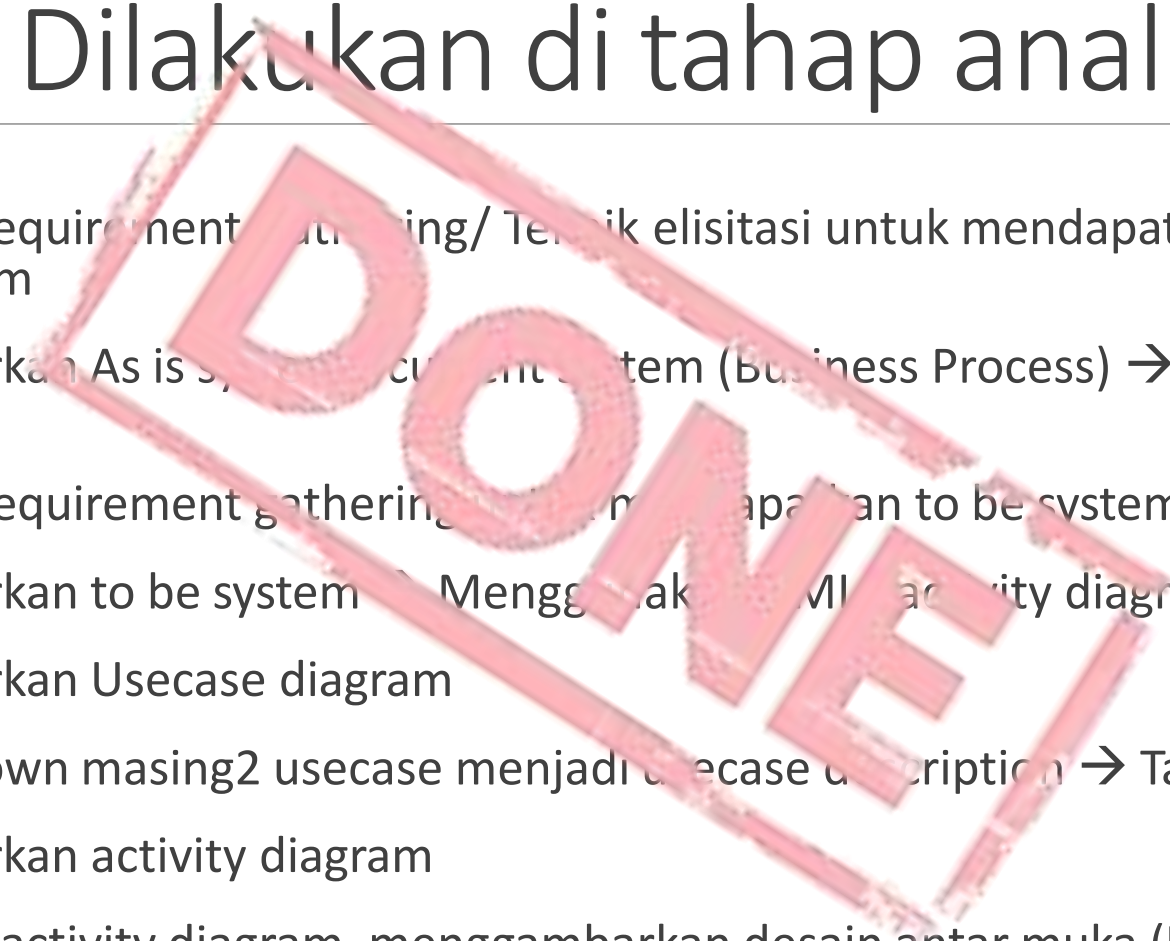
# Target Pembahasan

1. Review SDLC
2. Merancang Struktur Perangkat Lunak
    1. Architectural style perangkat Lunak
    2. Data-centered architectures
    3. Struktur umum dari rancangan perangkat lunak
    4. Sistem terdistribusi dari rancangan perangkat lunak
    5. Database architecture
    6. Data-flow architecture d
    7. Operating systems
3. Hardware Interface (Antarmuka Perangkat Keras)
4. Software Interface (Antarmuka Perangkat Lunak)
5. Communication Interface (Antarmuka Komunikasi)

# Business Process Analysis Steps



# Yang Dilakukan di tahap analisa




1. Melakukan Requirement gathering/ Teknik elisitasi untuk mendapatkan gambaran proses bisnis dari current system
2. Menggambarkan As is system (Business Process) → Menggunakan UML : activity diagram
3. Melakukan Requirement gathering untuk mendapatkan to be system
4. Menggambarkan to be system → Menggunakan UML activity diagram
5. Menggambarkan Usecase diagram
6. Membreakdown masing2 usecase menjadi usecase description → Tahap ini bisa dilewati
7. Menggambarkan activity diagram
8. Untuk setiap activity diagram, menggambarkan desain antar muka (User Interface)

# Yang Dilakukan di tahap Desain

1. Dari Activity diagram dan desain antar muka yang dibuat di tahap analisa, dilakukan breakdown menjadi Sequence diagram
2. Menggambarkan Class Diagram
3. Melakukan Permodelan data
4. Mendesain User Interface (User Experience)
5. Mendesain Arsitektur sistem

# SKKNI YANG AKAN DIBAHAS

Kode Unit : J.62SAD00.012.1

Judul : Merancang Struktur Perangkat Lunak

Deskripsi Unit : Unit kompetensi ini berhubungan dengan pengetahuan, keterampilan, dan sikap yang dibutuhkan dalam merancang komponen perangkat lunak.

Elemen Unit :

1. Mendefinisikan Struktur perangkat lunak
2. Menyusun Struktur perangkat lunak

# Target Pembahasan

1. Review SDLC
2. Merancang Struktur Perangkat Lunak
    1. Architectural style perangkat Lunak
    2. Data-centered architectures
    3. Struktur umum dari rancangan perangkat lunak
    4. Sistem terdistribusi dari rancangan perangkat lunak
    5. Database architecture
    6. Data-flow architecture d
    7. Operating systems
3. Hardware Interface (Antarmuka Perangkat Keras)
4. Software Interface (Antarmuka Perangkat Lunak)
5. Communication Interface (Antarmuka Komunikasi)

# Merancang Struktur Perangkat Lunak

Dalam SKKNI, anda diminta untuk dapat Mengidentifikasi / Mendefinisikan dan Merancang Struktur Perangkat Lunak di bawah ini :

1. Architectural style perangkat Lunak
2. Data-centered architectures
3. Struktur umum dari rancangan perangkat lunak
4. Sistem terdistribusi dari rancangan perangkat lunak
5. Database architecture
6. Data-flow architecture
7. Operating systems

# Architectural Style dalam SKKNI

Architectural style adalah spesialisasi tipe elemen dan relasi beserta konstrainnya, seperti:

- General structures (misal: **layers**, pipes and filters, blackboard);
- Distributed systems (misal: clientserver, three tiers, broker);
- Interactive systems (misal: model-view-controller, presentation-abstraction-control);
- Adaptable Systems (missal :microkernel, reflection);

# Layered Architecture (Arsitektur Bertingkat)

Arsitektur bertingkat membagi sistem perangkat lunak menjadi beberapa lapisan atau tingkat (layers) yang masing-masing memiliki tanggung jawab tertentu. Setiap lapisan hanya berinteraksi dengan lapisan di atas atau di bawahnya, mengisolasi tanggung jawab dan memberikan fleksibilitas serta kemudahan dalam pemeliharaan.

Komponen:

- Presentation Layer: Menyediakan antarmuka pengguna (UI) dan menangani interaksi pengguna.
- Application Layer: Bertanggung jawab atas logika aplikasi dan aturan bisnis.
- Data Layer: Mengelola akses dan penyimpanan data (biasanya ke database).
- Infrastructure Layer: Menangani masalah teknis seperti keamanan dan konektivitas.

# Pipes and Filters

Gaya arsitektur **Pipes and Filters** terdiri dari komponen yang disebut **filters** yang memproses data, dan **pipes** yang menghubungkan filter-filter ini, memungkinkan aliran data dari satu komponen ke komponen lainnya. Ini adalah model yang sangat sesuai untuk aliran data atau transformasi data bertahap, di mana data diproses dalam urutan yang terdefinisi oleh filter-filter.

**Komponen**:

- **Filter**: Setiap filter bertanggung jawab untuk melakukan satu unit pemrosesan pada data, seperti filter yang mengubah format data atau melakukan perhitungan.
- **Pipe**: Pipe adalah saluran yang mentransfer data antara filter-filter. Pipe menghubungkan satu filter dengan yang lainnya dalam urutan tertentu.

# Blackboard Architecture (Arsitektur Blackboard)

Arsitektur **Blackboard** adalah gaya arsitektur di mana **komponen-komponen independen** (sering disebut **knowledge sources**) berinteraksi dengan **blackboard**, yaitu tempat penyimpanan bersama yang digunakan oleh semua komponen untuk bertukar informasi. Model ini sangat cocok untuk masalah yang kompleks, di mana beberapa solusi atau pendapat dari berbagai komponen perlu digabungkan untuk mencapai hasil yang optimal.

**Komponen**:

- **Knowledge Sources**: Setiap knowledge source adalah komponen independen yang memiliki pengetahuan atau kemampuan tertentu untuk memecahkan sebagian masalah.
- **Blackboard**: Tempat penyimpanan yang digunakan untuk berbagi data dan informasi antar komponen. Data yang ditempatkan di blackboard akan diproses dan dimodifikasi oleh knowledge sources sesuai dengan algoritma atau aturan yang mereka miliki.

# Pemilihan Arsitektur untuk General Structures

- Layered Architecture: Cocok untuk aplikasi yang membutuhkan pemisahan logika bisnis, antarmuka pengguna, dan data. Ini sangat berguna di aplikasi besar dan kompleks di mana modularitas dan pemeliharaan penting.
- Pipes and Filters: Ideal untuk pemrosesan data bertahap atau aliran data yang harus diproses melalui berbagai tahap. Banyak digunakan dalam aplikasi yang memerlukan transformasi data atau pemrosesan stream.
- Blackboard Architecture: Digunakan untuk masalah yang kompleks dan beragam, di mana berbagai komponen berinteraksi dengan data bersama untuk menemukan solusi terbaik. Biasanya diterapkan dalam sistem AI atau sistem pakar.

# Architectural Style dalam SKKNI

Architectural style adalah spesialisasi tipe elemen dan relasi beserta konstrainnya, seperti:

- General structures (misal: **layers**, pipes and filters, blackboard);
- Distributed systems (misal: **clientserver, three tiers**, broker);
- Interactive systems (misal: model-view-controller, presentation-abstraction-control);
- Adaptable Systems (missal :microkernel, reflection);

# Client-Server Architecture

Arsitektur Client-Server adalah model paling dasar dan sering digunakan dalam sistem terdistribusi. Dalam model ini, ada dua peran utama: client dan server.

- Client adalah perangkat atau aplikasi yang mengirimkan permintaan untuk layanan atau data.
- Server adalah perangkat atau aplikasi yang menyediakan layanan atau data yang diminta oleh client.

Cara Kerja:

- Client mengirimkan permintaan (request) ke server melalui jaringan.
- Server memproses permintaan tersebut dan mengirimkan respon (response) kembali ke client.
- Server biasanya memiliki sumber daya yang lebih kuat (seperti database atau aplikasi) dan mengelola permintaan dari banyak client secara bersamaan.

# Three-Tier Architecture

Arsitektur Three-Tier (Tiga Lapisan) adalah pengembangan dari model Client-Server yang membagi aplikasi menjadi tiga lapisan utama: presentation layer, application layer, dan data layer.

Lapisan dalam Three-Tier:

1. Presentation Layer: Lapisan ini bertanggung jawab untuk antarmuka pengguna (UI). Ini adalah lapisan yang berinteraksi langsung dengan pengguna.
   Misalnya, aplikasi web atau mobile yang menampilkan informasi kepada pengguna.
2. Application Layer: Lapisan ini mengandung logika bisnis dan aturan yang mendasari aplikasi. Ini berfungsi untuk memproses permintaan dan interaksi antar data.
   Ini adalah lapisan yang menangani pemrosesan data dan pengolahan logika aplikasi.
3. Data Layer: Lapisan ini bertanggung jawab untuk penyimpanan data, biasanya di database.
   Ini mengelola akses dan penyimpanan data dalam sistem.

# Broker Architecture

Arsitektur Broker digunakan dalam sistem terdistribusi yang membutuhkan komunikasi antara komponen-komponen yang saling berinteraksi, tetapi mungkin tidak saling mengetahui atau tidak terhubung secara langsung. Broker bertindak sebagai perantara yang memungkinkan komunikasi antara client dan server atau antara berbagai komponen dalam sistem.

Komponen Utama:

1. Broker: Komponen yang bertanggung jawab untuk menerima permintaan dari client dan meneruskannya ke server yang sesuai untuk diproses.
2. Server: Komponen yang menyediakan layanan yang diminta oleh client.
3. Client: Komponen yang mengirimkan permintaan layanan.

# Ringkasan dan Pemilihan Model yang Tepat

- **Client-Server**: Cocok untuk aplikasi yang sederhana di mana komunikasi antara client dan server tidak memerlukan banyak lapisan atau perantara.
- **Three-Tier**: Ideal untuk aplikasi yang lebih kompleks dengan kebutuhan untuk memisahkan lapisan tampilan, logika aplikasi, dan data, memberikan modularitas dan skalabilitas yang lebih baik.
- **Broker**: Paling sesuai untuk sistem terdistribusi yang kompleks dengan banyak komponen yang saling berinteraksi namun tidak langsung saling terhubung. Broker menyediakan lapisan perantara yang memungkinkan komponen yang terpisah untuk berkomunikasi tanpa mengetahui detil satu sama lain.

# Architectural Style dalam SKKNI

Architectural style adalah spesialisasi tipe elemen dan relasi beserta konstrainnya, seperti:

- General structures (misal: **layers**, pipes and filters, blackboard);
- Distributed systems (misal: **clientserver, three tiers**, broker);
- Interactive systems (misal: **model-view-controller**, presentation-abstraction-control);
- Adaptable Systems (missal :microkernel, reflection);

# Interactive systems

adalah sistem perangkat lunak yang melibatkan interaksi langsung antara pengguna dan sistem, biasanya melalui antarmuka pengguna (UI).

Tujuan utama dari sistem interaktif adalah untuk memberikan pengalaman pengguna yang efektif, efisien, dan intuitif.

Arsitektur atau design patterns yang digunakan untuk mengelola interaksi antara berbagai komponen dalam sistem ini sangat penting untuk memastikan desain yang modular, fleksibel, dan mudah dipelihara.

# Model-View-Controller (MVC)

**Model-View-Controller (MVC)** adalah salah satu pola desain yang paling terkenal dan digunakan untuk memisahkan aplikasi menjadi tiga komponen utama: **Model**, **View**, dan **Controller**. **Komponen dalam MVC**:

1. **Model**:
    1. **Tugas**: Mengelola data dan logika bisnis aplikasi. Model bertanggung jawab untuk mengambil, memproses, dan menyimpan data, serta memberikan data yang relevan ke View.
    2. **Fungsi**: Mengatur interaksi dengan database atau sumber data lainnya, serta aturan logika yang digunakan untuk mengubah atau memvalidasi data.

2. **View**:
    1. **Tugas**: Menampilkan data atau informasi kepada pengguna. View bertanggung jawab untuk rendering antarmuka pengguna berdasarkan data yang diterima dari Model.
    2. **Fungsi**: Mengambil data dari Model dan menampilkannya kepada pengguna dalam bentuk yang mudah dipahami, misalnya melalui antarmuka grafis atau web.

3. **Controller**:
    1. **Tugas**: Bertindak sebagai penghubung antara Model dan View. Controller menerima input dari pengguna, memprosesnya (mengubah data di Model), dan memperbarui View untuk menampilkan hasil yang sesuai.
    2. **Fungsi**: Mengontrol aliran informasi antara Model dan View, serta menangani logika aplikasi yang tidak termasuk dalam Model atau View.

# Presentation-Abstraction-Control (PAC)

Presentation-Abstraction-Control (PAC) adalah pola desain yang lebih kompleks dibandingkan dengan MVC dan lebih banyak digunakan dalam sistem terdistribusi atau sistem yang memerlukan pengelolaan antarmuka dan data secara lebih dinamis dan terorganisir. PAC juga merupakan pola yang lebih terstruktur dalam memisahkan tanggung jawab antara presentasi, abstraksi, dan kontrol.

Komponen dalam PAC:

1. Presentation:
    1. Tugas: Menangani antarmuka pengguna. Presentation bertanggung jawab untuk menunjukkan data kepada pengguna dalam bentuk visual atau grafis.
    2. Fungsi: Ini adalah bagian yang berinteraksi langsung dengan pengguna dan menangani input dari pengguna, serta memberikan tampilan atau representasi dari data.
2. Abstraction:
    1. Tugas: Mengelola logika bisnis dan data. Abstraction bertanggung jawab untuk menyimpan dan mengelola data aplikasi serta menangani pemrosesan atau manipulasi data.
    2. Fungsi: Ini adalah bagian yang mengakses sumber daya eksternal (seperti database atau layanan lain), serta melakukan operasi pada data yang dibutuhkan oleh aplikasi.
3. Control:
    1. Tugas: Mengontrol aliran informasi dan interaksi antara Presentation dan Abstraction.
    2. Fungsi: Control bertanggung jawab untuk mengelola interaksi antara pengguna dan sistem, menentukan kapan dan bagaimana data dari Abstraction akan diperbarui atau ditampilkan oleh Presentation.

# Perbandingan MVC dan PAC

- Sederhana vs. Kompleksitas: MVC lebih sederhana dan lebih mudah diimplementasikan, sedangkan PAC lebih fleksibel dan lebih cocok untuk aplikasi besar dan sistem terdistribusi yang membutuhkan pemisahan lebih rinci antara komponen.
- Modularitas: Keduanya mendukung modularitas, tetapi PAC menyediakan tingkat pemisahan yang lebih besar dan lebih banyak kontrol atas interaksi antar komponen.
- Penggunaan: MVC lebih banyak digunakan dalam aplikasi web dan aplikasi berbasis desktop dengan UI sederhana, sementara PAC lebih banyak digunakan dalam sistem yang lebih kompleks, seperti sistem berbasis perangkat atau aplikasi yang memerlukan pembagian tugas yang lebih besar.

# Architectural Style dalam SKKNI

Architectural style adalah spesialisasi tipe elemen dan relasi beserta konstrainnya, seperti:

- General structures (misal: layers, pipes and filters, blackboard);
- Distributed systems (misal: clientserver, three tiers, broker);
- Interactive systems (misal: model-view-controller, presentation-abstraction-control);
- Adaptable Systems (missal :microkernel, reflection);

# Adaptable systems

Adaptable systems adalah sistem perangkat lunak yang dirancang untuk dapat dengan mudah beradaptasi atau berubah sesuai dengan perubahan kebutuhan, kondisi lingkungan, atau teknologi.

Sistem ini sangat penting dalam pengembangan perangkat lunak modern karena kebutuhan bisnis dan teknologi sering berubah, sehingga sistem yang dapat beradaptasi akan lebih tahan lama dan fleksibel.

# Microkernel Architecture

Arsitektur Microkernel adalah pola desain di mana sistem utama dibangun dengan sebuah kernel kecil yang sangat ringan, sementara fungsionalitas tambahan atau ekstensi disediakan oleh modul-modul terpisah yang berkomunikasi dengan kernel melalui antarmuka yang terdefinisi dengan jelas. Dalam arsitektur ini, kernel bertanggung jawab hanya untuk layanan dasar seperti pengelolaan sumber daya dan komunikasi antar modul, sementara fitur-fitur tambahan dapat ditambahkan atau diubah tanpa mempengaruhi kernel utama.

Komponen dalam Microkernel:

1. Kernel:
   - Bertanggung jawab untuk pengelolaan sumber daya dasar (seperti manajemen proses, komunikasi antar modul, manajemen memori).
   - Kernel sangat kecil dan hanya menyediakan fungsi dasar yang diperlukan oleh sistem.

2. Modul Ekstensi:
   - Berfungsi sebagai tambahan fitur atau layanan yang dapat diaktifkan atau dinonaktifkan sesuai kebutuhan. Modul ini bisa mencakup fungsionalitas seperti antarmuka pengguna (UI), protokol jaringan, atau modul lainnya yang dapat dimuat sesuai permintaan.
   - Modul-modul ini dapat dikembangkan secara independen dan ditambahkan atau diganti dengan mudah tanpa mempengaruhi kernel.

# Reflection Architecture

Reflection adalah mekanisme dalam perangkat lunak yang memungkinkan sistem untuk memeriksa, mengubah, dan beradaptasi dengan struktur dan perilaku dirinya sendiri pada waktu berjalan (runtime). Dengan menggunakan reflection, sistem dapat menyesuaikan dirinya dengan keadaan baru atau memperbarui dirinya sendiri untuk mendukung kebutuhan baru, bahkan tanpa harus memodifikasi kode sumber.

Komponen dalam Reflection:

1. Reflection API:
   - Biasanya, ada sebuah API khusus yang memungkinkan program untuk melakukan introspeksi (melihat struktur internal) dan modifikasi (mengubah struktur atau perilaku) objek atau kelas pada waktu berjalan.
   - Reflection memungkinkan aplikasi untuk mengeksekusi fungsi atau memanggil metode tanpa mengetahui detail implementasi pada waktu kompilasi.

2. Metadata:
   - Reflection memanfaatkan metadata (data tentang data) untuk menentukan struktur kelas atau objek dalam program. Ini termasuk informasi seperti nama kelas, parameter metode, tipe data, dan atribut.

# 1. Architectural Style secara Umum

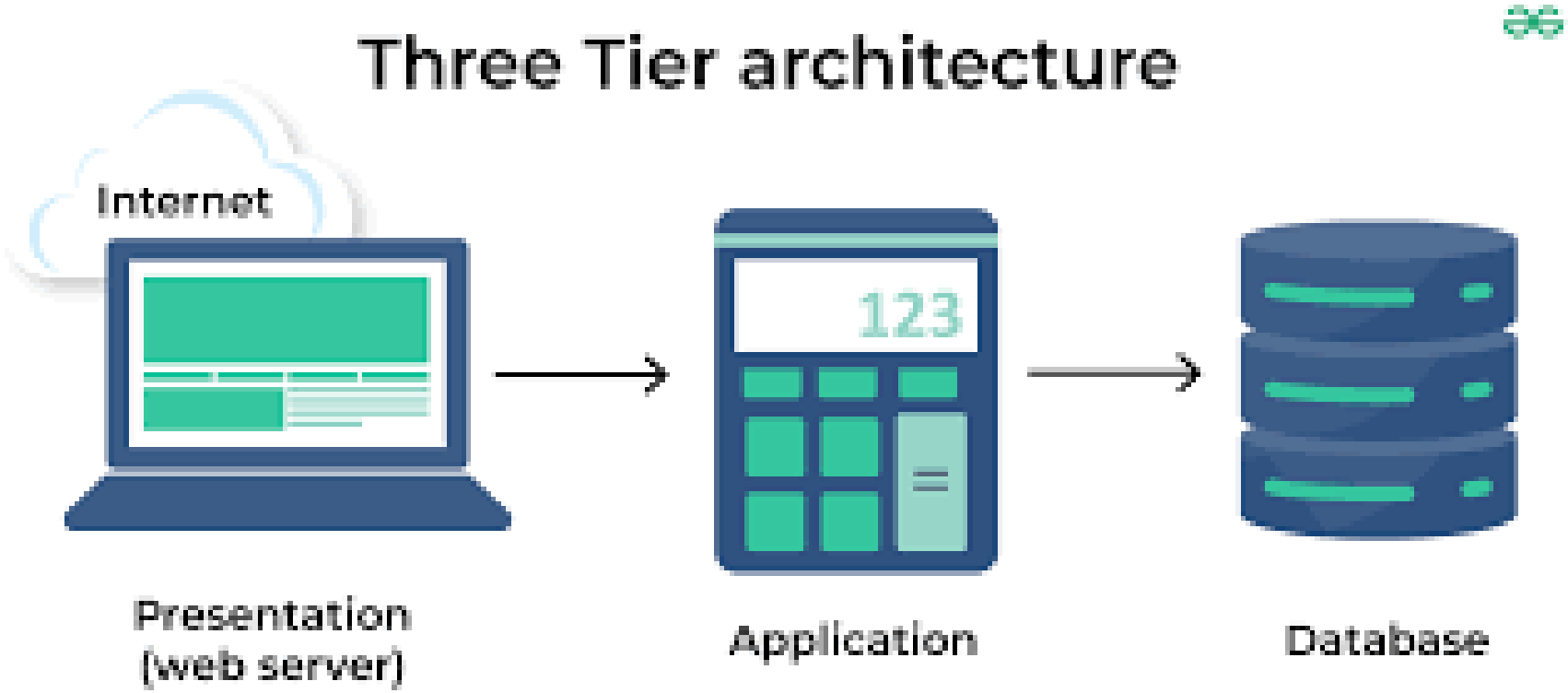
Merupakan pola umum atau pendekatan desain tingkat tinggi yang digunakan untuk mengatur struktur dan komponen utama dari perangkat lunak.

| Jenis Arsitektur | Penjelasan Singkat |
|---|---|
| **Layered (Bertingkat)** | Sistem dibagi menjadi lapisan-lapisan (misalnya: Presentation, Business Logic, Data Access). |
| **Client-Server** | Ada dua entitas utama: *client* yang meminta layanan, dan *server* yang menyediakan layanan. |
| **Microservices** | Aplikasi dibangun dari kumpulan layanan kecil yang berjalan secara independen dan saling berkomunikasi. |
| **Service-Oriented Architecture (SOA)** | Layanan-layanan independen yang dapat digunakan ulang dan diintegrasikan. |

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Jenis Gaya Arsitektur | **Layered Architecture (3-Layer Architecture)** |
| Alasan Pemilihan | Memisahkan tampilan, logika bisnis, dan data untuk memudahkan pengembangan dan pemeliharaan. |
| Komponen Utama | - **Presentation Layer**: Antarmuka pengguna (UI) berbasis web.<br>- **Business Logic Layer**: Proses peminjaman, pengembalian, validasi data.<br>- **Data Access Layer**: Koneksi dan query ke database. |

# Contoh Pada Sistem Perpustakaan



# Target Pembahasan

1. Review SDLC
2. Merancang Struktur Perangkat Lunak
    1. Architectural style perangkat Lunak
    2. Data-centered architectures
    3. Struktur umum dari rancangan perangkat lunak
    4. Sistem terdistribusi dari rancangan perangkat lunak
    5. Database architecture
    6. Data-flow architecture d
    7. Operating systems
3. Hardware Interface (Antarmuka Perangkat Keras)
4. Software Interface (Antarmuka Perangkat Lunak)
5. Communication Interface (Antarmuka Komunikasi)

# 2. Data-centered architectures

Arsitektur yang berfokus pada data sebagai pusat komunikasi antar komponen sistem. Semua proses berbagi dan memodifikasi data dilakukan melalui pusat data bersama (repository atau blackboard).

- Database-Centered System → semua aplikasi mengakses data dari satu database utama.
- Blackboard System → model desain yang mendorong kolaborasi antar-komponen khusus—sering disebut sumber pengetahuan atau pakar —untuk memecahkan masalah. (AI Based)

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Model Arsitektur Data | **Repository Model (Database-Centered System)** |
| Deskripsi | Semua modul (peminjaman, pengembalian, laporan) mengakses dan memperbarui data melalui satu basis data pusat. |
| Komponen Data Utama | - Tabel Buku, Tabel Anggota, Tabel Peminjaman, Tabel Pengembalian, Tabel Petugas |
| Diagram Pendukung | Diagram ERD / Diagram Class UML |

# 3. Struktur umum dari rancangan perangkat lunak

Merupakan kerangka dasar yang menggambarkan hubungan antar komponen perangkat lunak dan interaksi di dalamnya.

Isi Struktur Umum Biasanya Meliputi:

1. Komponen utama (modules, classes, subsystems)
2. Hubungan antar komponen (dependency, communication path)
3. Pola interaksi (synchronous/asynchronous)
4. Layer dan boundary system

Contoh Dokumen: Diagram arsitektur, diagram komponen UML, atau diagram paket.

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Komponen Utama | Modul Login, Modul Data Buku, Modul Data Anggota, Modul Peminjaman & Pengembalian, Modul Laporan |
| Hubungan Antar Komponen | Semua modul diakses melalui menu utama sistem dan terhubung dengan database melalui *data access layer*. |
| Pola Desain yang Digunakan | **Model-View-Controller (MVC)** |
| Diagram | Lampirkan diagram komponen UML / package diagram / diwakili dengan Sequence yang menunjukkam model MVC |

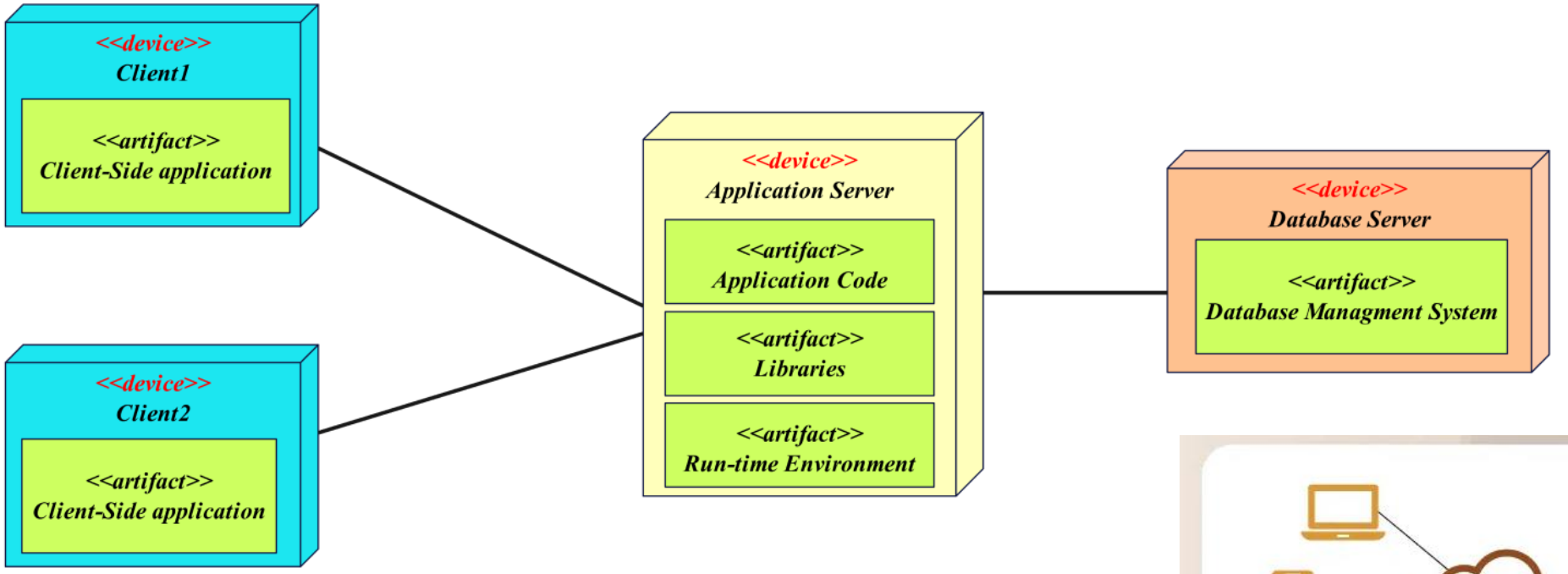
# 4. Sistem terdistribusi dari rancangan perangkat lunak

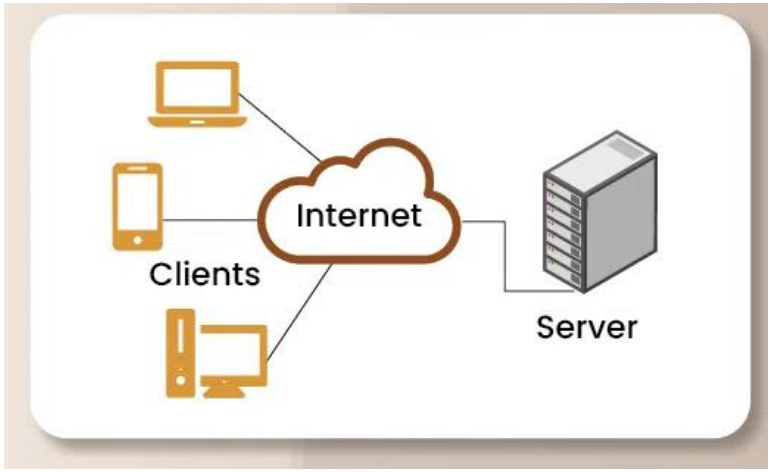
Arsitektur di mana komponen sistem dijalankan di beberapa komputer (node) yang saling terhubung melalui jaringan.Contoh Model Sistem Terdistribusi:

| Model | Penjelasan |
|---|---|
| **Client-Server** | Komponen client dan server tersebar pada node berbeda. |
| **Peer-to-Peer (P2P)** | Tiap node dapat bertindak sebagai client sekaligus server. |
| **3-Tier / N-Tier** | Terdiri dari lapisan presentasi, logika bisnis, dan database yang dijalankan di server berbeda. |
| **Cloud-Based Architecture** | Sistem didistribusikan pada layanan cloud (microservices, container, API gateway). |

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Jenis Sistem Terdistribusi | **Client-Server Architecture (Web-Based)** |
| Deskripsi Distribusi Komponen | Client mengakses aplikasi web melalui browser, sedangkan server menjalankan aplikasi dan database. |
| Komunikasi Antar Node | Menggunakan protokol **HTTP/HTTPS** untuk komunikasi data. |
| Diagram Deployment | (Lampirkan diagram deployment UML: client – web server – database server) |


<<device>>
Client1
<<artifact>>
Client-Side application
<<device>>
Client2
<<artifact>>
Client-Side application
<<device>>
Application Server
<<artifact>>
Application Code
<<artifact>>
Libraries
<<artifact>>
Run-time Environment
<<device>>
Database Server
<<artifact>>
Database Managment System


Internet
Clients
Server

# 5. Database architecture

Menentukan struktur, organisasi, dan hubungan antar data dalam sistem, serta bagaimana data diakses dan dikelola oleh aplikasi.

| Jenis | Penjelasan |
|---|---|
| **Centralized DB** | Semua data tersimpan di satu lokasi pusat. |
| **Distributed DB** | Data tersebar di beberapa lokasi namun dikelola secara terkoordinasi. |
| **Client-Server DB** | DBMS berjalan di server dan diakses oleh aplikasi client. |
| **Cloud Database** | Database di-host di lingkungan cloud dengan skalabilitas tinggi. |

**Contoh desain:** ERD (Entity Relationship Diagram), skema tabel, relasi antar tabel. / Class Diagram untuk OOAD

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Jenis Arsitektur Database | **Client-Server Database Architecture** |
| DBMS yang Digunakan | **MySQL / MariaDB** |
| Struktur Database | - Buku (id_buku, judul, pengarang, tahun, kategori)- Anggota (id_anggota, nama, alamat, no_hp)- Peminjaman (id_pinjam, id_buku, id_anggota, tgl_pinjam, tgl_kembali, status)- Pengembalian (id_kembali, id_pinjam, denda) |
| Diagram Yang ditunjukkan | (Lampirkan ERD dengan relasi antar tabel) atau Class Diagram untuk OOAD |

# 6. Data-flow architecture

Arsitektur yang menekankan **alur data** dari input → proses → output.

Biasanya digambarkan dalam bentuk **Data Flow Diagram (DFD)** atau arsitektur berbasis *pipe and filter*.

**Contoh:**

1. Sistem pemrosesan data sensor
2. Aplikasi ETL (Extract, Transform, Load)
3. Pipeline pengolahan citra atau data streaming

**Ciri khas:**

- Data mengalir dari satu komponen ke komponen lain.
- Setiap komponen melakukan transformasi terhadap data.

# Perbandingan UML dan DFD dalam Menggambarkan Data Flow Architecture

| Aspek | UML | DFD (Data Flow Diagram) |
|---|---|---|
| **Fokus Utama** | Menunjukkan **struktur objek** dan **interaksi** antar objek. | Menunjukkan **aliran data** dan bagaimana data diproses. |
| **Tujuan** | Pemodelan objek, alur interaksi, dan struktur sistem. | Menggambarkan **proses data** dan hubungan antar proses. |
| **Diagram yang Digunakan** | Usecase Diagram, Activity Diagram, Sequence Diagram, Class Diagram. | Data Flow Diagram (DFD) |
| **Detail Aliran Data** | Tidak mendetailkan aliran data secara eksplisit. | **DFD** menggambarkan aliran data dengan sangat rinci. |
| **Penggunaan** | Pemodelan **interaksi** objek dan **proses** dalam konteks sistem yang lebih besar. | Pemodelan **proses** dan aliran data dalam aplikasi atau sistem tertentu. |
| **Modularitas dan Pembagian Proses** | Bisa menggambarkan interaksi antar modul atau komponen. | Secara eksplisit membagi sistem menjadi **proses-proses** terpisah yang saling berinteraksi. |

# Perbedaan DFD dan Class Diagram

| Aspek | Data-Flow Architecture (DFD) | Class Diagram (UML) |
|---|---|---|
| **Tujuan Utama** | Menunjukkan bagaimana **data** bergerak dan diproses dalam sistem. | Menunjukkan **struktur objek**, **relasi antar kelas**, dan **atribut/metode**. |
| **Fokus** | Aliran data, proses, dan penyimpanan data. | Struktur dan hubungan antar kelas dalam sistem. |
| **Komponen Utama** | Proses, aliran data, penyimpanan data, entitas eksternal. | Kelas, atribut, metode, asosiasi antar kelas. |
| **Level Abstraksi** | Lebih berfokus pada **fungsi** dan aliran data dalam sistem. | Lebih berfokus pada **struktur statis** dan relasi antar entitas. |
| **Contoh Penggunaan** | Sistem berbasis aliran data (misalnya ETL, sistem pemrosesan batch). | Desain sistem berorientasi objek (misalnya aplikasi berbasis objek). |

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Jenis Arsitektur Aliran Data | **Data Flow Diagram (DFD)** |
| Alur Data Utama | Input dari pengguna → validasi data → simpan ke database → tampilkan laporan/output. |
| Komponen Pemrosesan Data | - Proses login- Proses manajemen data buku- Proses transaksi peminjaman/pengembalian- Proses pembuatan laporan |
| Diagram Aliran Data | DFD |

# 7. Operating systems

Analis sistem perlu **memahami lingkungan sistem operasi** yang akan digunakan agar rancangan perangkat lunak sesuai dengan platform target.

**Aspek yang perlu didefinisikan:**

1. Sistem operasi target (Windows, Linux, macOS, Android, iOS)
2. Dukungan terhadap multi-user, multitasking, dan keamanan
3. Integrasi dengan sistem file, jaringan, atau API OS
4. Kebutuhan kompatibilitas dan portabilitas perangkat lunak

# Contoh Pada Sistem Perpustakaan

| Elemen | Isi Deskripsi |
|---|---|
| Sistem Operasi Target | **Windows Server 2022 / Linux Ubuntu Server 22.04** |
| Lingkungan Eksekusi | **Web Server (Apache / Nginx)** dengan **PHP 8.x** dan **MySQL** |
| Ketergantungan OS | Library PHP, ekstensi database mysqli, serta browser modern (Chrome, Firefox). |
| Kebutuhan Portabilitas | Sistem dapat dijalankan di server lokal atau cloud. |

# Target Pembahasan

1. Review SDLC
2. Merancang Struktur Perangkat Lunak
    1. Architectural style perangkat Lunak
    2. Data-centered architectures
    3. Struktur umum dari rancangan perangkat lunak
    4. Sistem terdistribusi dari rancangan perangkat lunak
    5. Database architecture
    6. Data-flow architecture d
    7. Operating systems
3. Hardware Interface (Antarmuka Perangkat Keras)
4. Software Interface (Antarmuka Perangkat Lunak)
5. Communication Interface (Antarmuka Komunikasi)

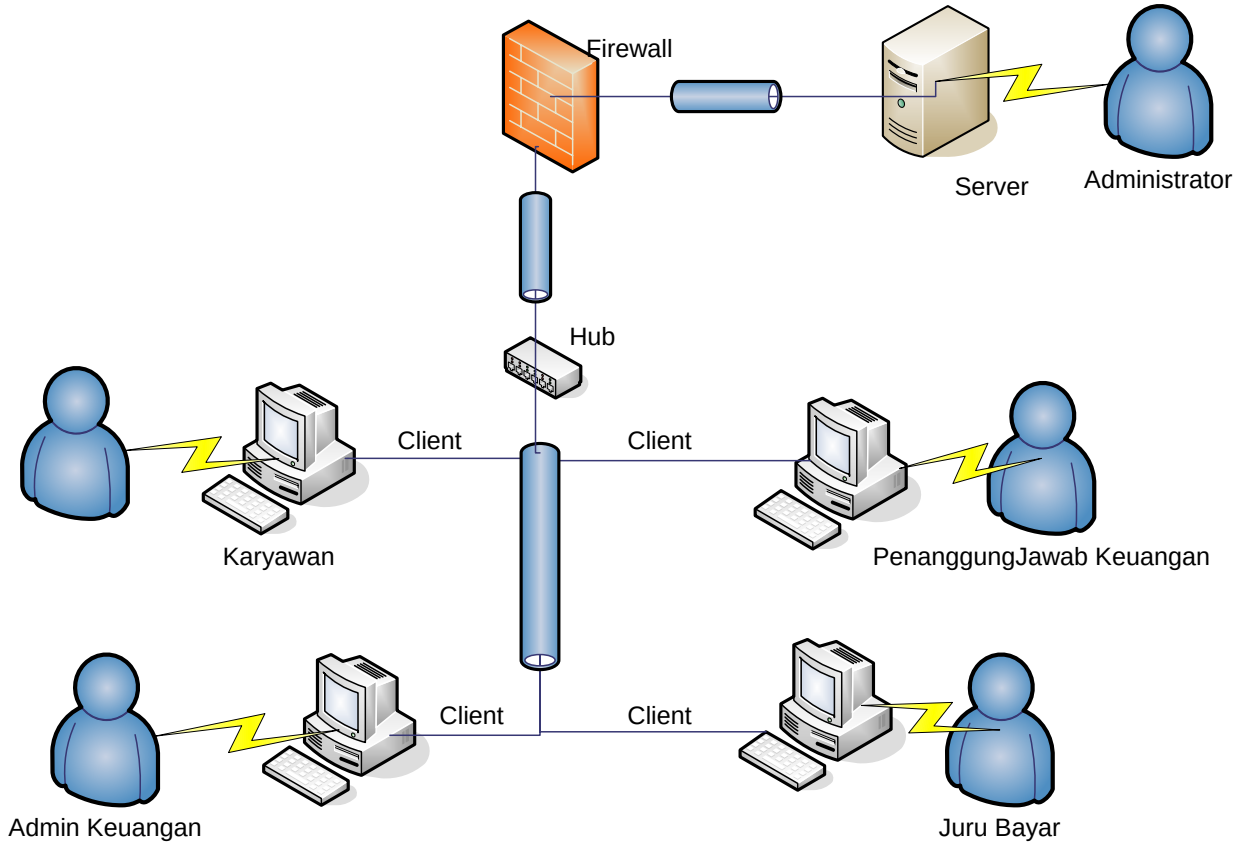
# Hardware Interface (Antarmuka Perangkat Keras)

Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system.

This may include :

- the supported device types,
- the nature of the data
- control interactions between the software and the hardware,
- **communication protocols to be used.**

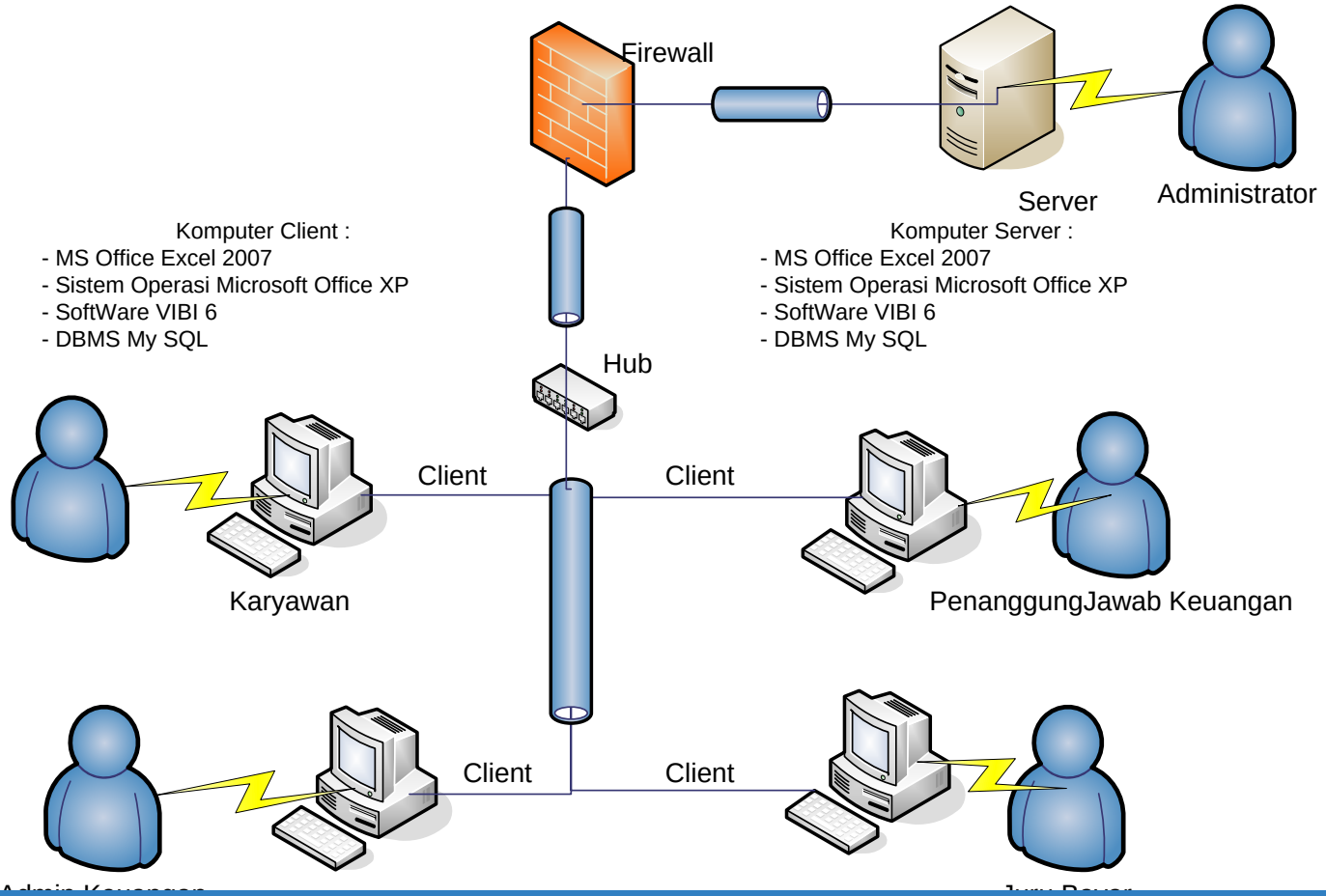
# Hardware Interface communication protocols to be used.



# Software Interface (Antarmuka Perangkat Lunak)

- **Describe the connections between this product and other specific software components (name and version), including databasesoperating systems, tools, libraries, and integrated**,.
- Identify the data items or messages coming into the system and going out and describe the purpocommercial componentsse of each.
- Describe the services needed and the nature of communications.
- Refer to documents that describe detailed application programming interface protocols.
- Identify data that will be shared across software components.
- If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint

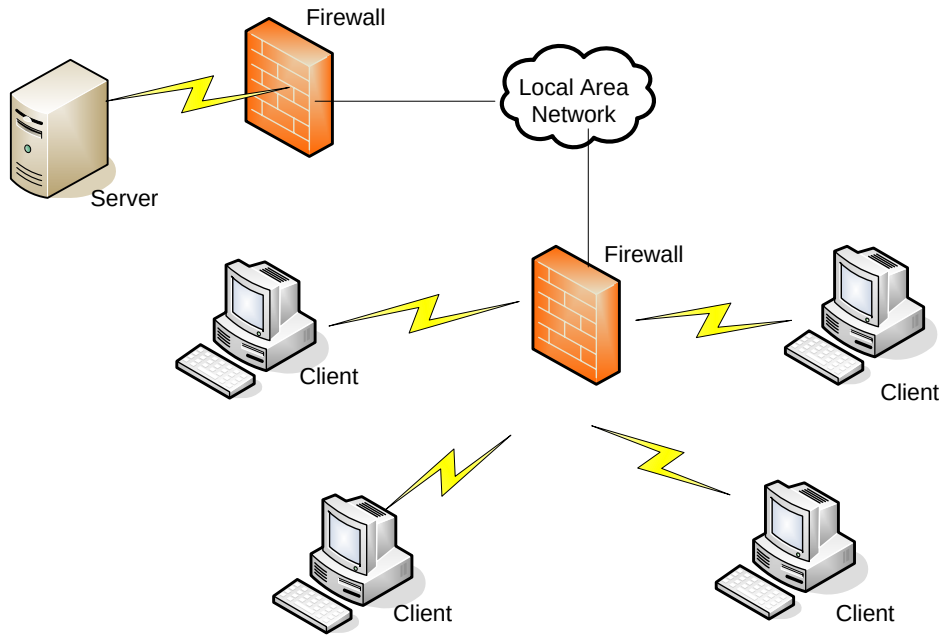
# Contoh Software Interface



# Communication Interface (Antarmuka Komunikasi)

- Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on.
- Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP.
- Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms

# Contoh Communication Interface

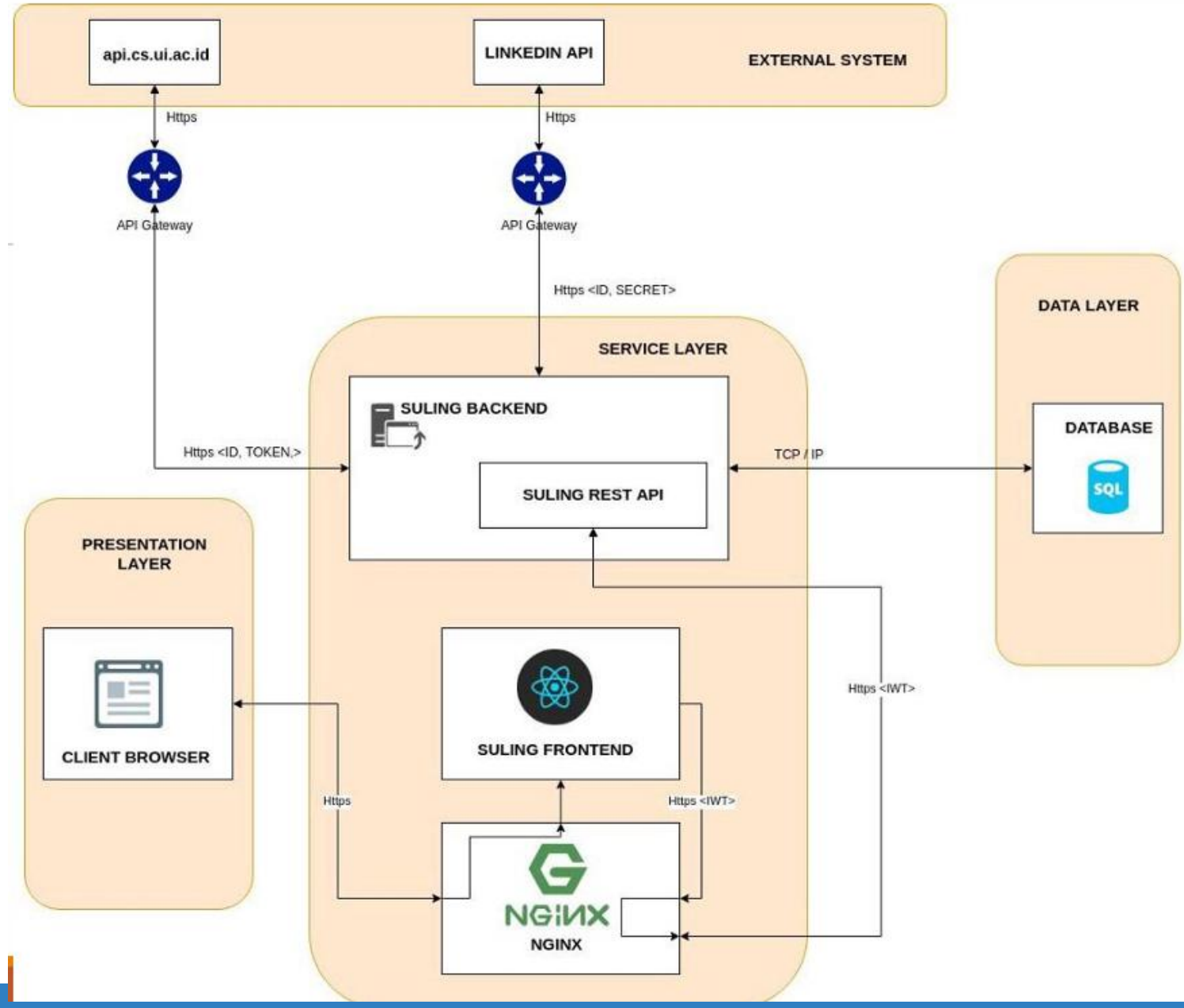


# Arsitektur Secara Umum

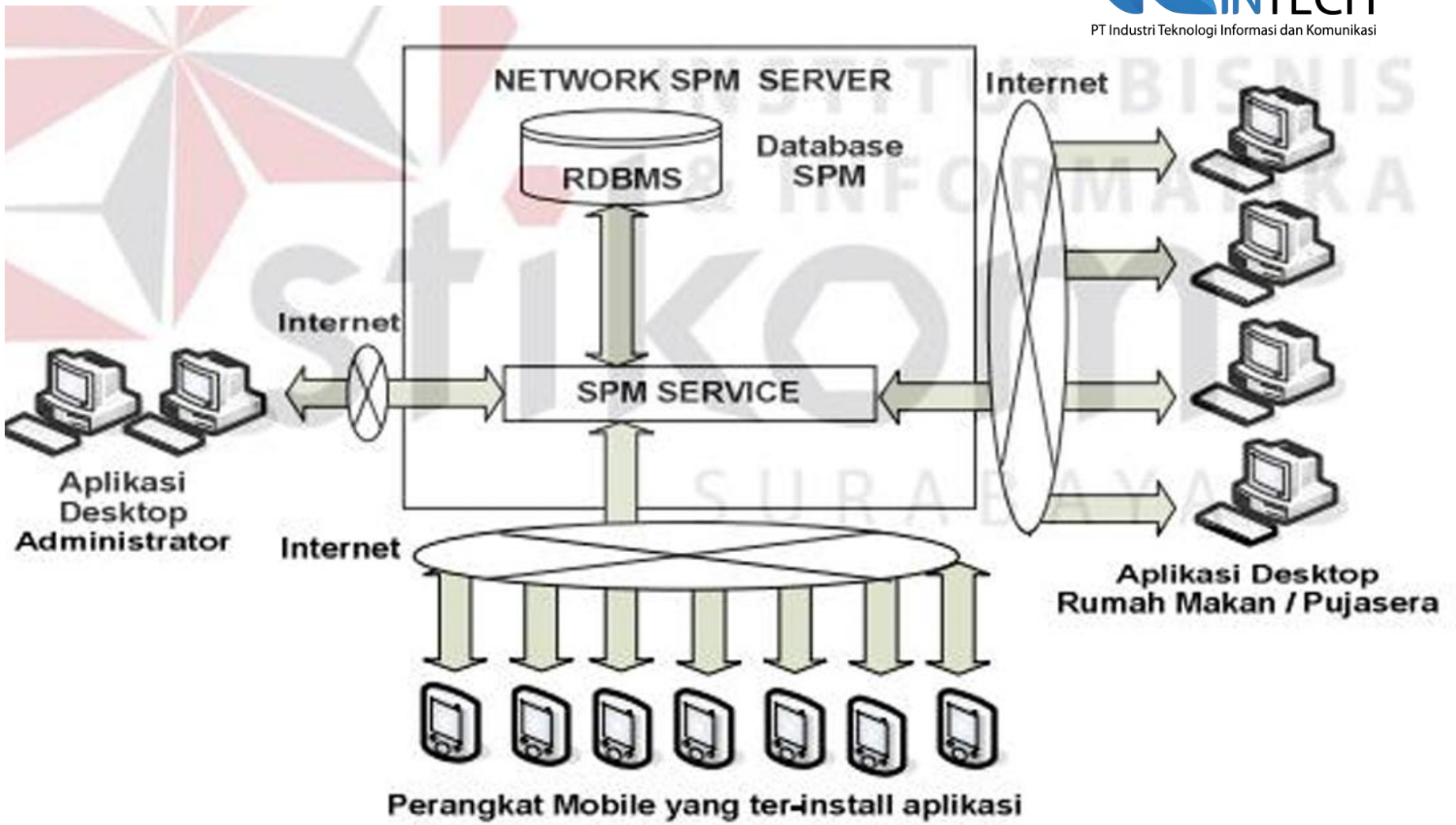
Menunjukkan Layer :

- Layer Aplikasi
- Layer Data
- Layer Presentasi
- Sistem eksternal
- Bagaimana saling berkomunikasi

# Contoh Desain Arsitektur



# Contoh Desain Arsitektur




thank you
danke
謝謝
ngiyabonga
teşekkür ederim
gracias
спасибо
dziękuję
bedankt
obrigado
merci
go raibh maith agat
mochchakkeram
tapadh leat
dank je
sukriya
kop khun krap
grazie
arigatô
dakujem
terima kasih
감사합니다
ευχαριστώ
мерси
баярлалаа
рахмат
hvala
mauruuru
sagolun
takk
taiku
dankie
kaitos
matondo
misaotra
blagodaram
paldies
asante
manana
mahalo
barka
faafetai
lava
vinaka
dhanyavadagalu
xiexie
shukriya
trugarez
merci